

SukukFi

Smart Contract Security Assessment

Audit dates: Nov 26 — Dec 05, 2025

Overview

About C4

Code4rena (C4) is a competitive audit platform where security researchers, referred to as Wardens, review, audit, and analyze codebases for security vulnerabilities in exchange for bounties provided by sponsoring projects.

During the audit outlined in this document, C4 conducted an analysis of the SukukFi smart contract system. The audit took place from November 26 to December 05, 2025.

Final report assembled by Code4rena.

Summary

The C4 analysis yielded an aggregated total of 4 unique vulnerabilities. Of these vulnerabilities, 1 received a risk rating in the category of HIGH severity and 3 received a risk rating in the category of MEDIUM severity.

Additionally, C4 analysis included 54 QA reports compiling issues with a risk rating of LOW severity or informational.

All of the issues presented here are linked back to their original finding, which may include relevant context from the judge and SukukFi team.

Scope

The code under review can be found within the [C4 SukukFi repository](#), and is composed of 6 smart contracts written in the Solidity programming language and includes 1,670 lines of Solidity code.

The code in C4's SukukFi repository was pulled from:

- Repository: <https://github.com/Engineering-Finance/erc7575>
- Commit hash: aec1ea2da86e1923fc0358a73823340bd42ae5da

Severity Criteria

C4 assesses the severity of disclosed vulnerabilities based on three primary risk categories: high, medium, and low/informational.

High-level considerations for vulnerabilities span the following key areas when conducting assessments:

- Malicious Input Handling
- Escalation of privileges

- Arithmetic
- Gas use

For more information regarding the severity criteria referenced throughout the submission review process, please refer to the documentation provided on [the C4 website](#), specifically our section on [Severity Categorization](#).

High Risk Findings (1)

[H-01] Missing Authorization Check Allows Unauthorized Fund Withdrawal

Submitted by [OxSeer](#), also found by [Orpse](#), [Ox04](#), [OxApple](#), [Oxbereket](#), [OxCryptOnite](#), [OxD4n13l](#), [OxDemon](#), [Oxdice91](#), [OxGOP1](#), [OxGutzzz](#), [Oxheartcode](#), [Oxkb](#), [OxMilenov](#), [Oxnija](#), [Oxpetern](#), [OxRaz](#), [Oxsai](#), [OxSecurious](#), [OxVrka](#), [Aamir](#), [abdelhaq16](#), [aestheticbhai](#), [Ahmerdrarerh](#), [ainta](#), [Albert](#), [aldarion](#), [ameng](#), [anchabadze](#), [anon1one](#), [arunabha003](#), [Audittens](#), [Avalance](#), [axelot](#), [ayushblock](#), [b_void](#), [Bbash](#), [blutorque](#), [Bobai23](#), [boraichodrunkenmaster](#), [boredpukar](#), [cOpp3rscr3w3r](#), [classic-k](#), [cosin3](#), [count-sum](#), [cyberEth](#), [dantehrani](#), [DarkWingCipher](#), [Dest1ny_rs](#), [Diavolo](#), [dimulski](#), [Dinesh11G](#), [djshan_edan](#), [dray](#), [duan](#), [edoscoba](#), [Egbe](#), [Ekene](#), [eloujoe](#), [EtherEngineer](#), [ETHworker](#), [Eurovickk](#), [Falendar](#), [farman1094](#), [fathomhewclaim](#), [freebird0323](#), [fullstop](#), [gabkov](#), [gbrigandi](#), [grigorovv](#), [Guilherme](#), [h2134](#), [happykilling](#), [hark017](#), [hezze](#), [hgrano](#), [hirusha](#), [hrmneffdij](#), [Insomnia](#), [itsravinOx](#), [j3x](#), [jerry0422](#), [jiangling](#), [jo13](#), [johnyfwesh](#), [Josh4324](#), [JuggerNaut63](#), [kimnoic](#), [KKKKK](#), [kmmk](#), [KuwaTakushi](#), [legendweb3](#), [lodelux](#), [LubuSeb](#), [makarov](#), [mbuba666](#), [meshaqRapha](#), [mrdaifidi](#), [mrMorningstar](#), [Mrunal2610](#), [nachin](#), [niffylord](#), [nutledger](#), [odeili](#), [osok](#), [Oxhsn](#), [pecata17107](#), [PentestMonkey](#), [phoenixV110](#), [prk0](#), [Psycharis](#), [pv](#), [radev_sw](#), [Ravi](#), [Ravisai](#), [renacoder](#), [richa](#), [Rikka](#), [romans](#), [RotiTelur](#), [rubencrxz](#), [s4bot3ur](#), [Sabit](#), [Samueltroydomi](#), [saraswati](#), [Shiraga94](#), [silver_eth](#), [Smacaud](#), [Sneks](#), [SOPROBRO](#), [soyaya](#), [Spektor](#), [Stador](#), [Supheli](#), [Synthrax](#), [Tarnished](#), [th3_hybrid](#), [TheMilG](#), [TOSHI](#), [trachev](#), [tradingview](#), [Uddercover](#), [Utsav](#), [v2110](#), [VAD37](#), [Valves](#), [vangrim](#), [wafflewizard](#), [Willy_Petrus](#), [wuji](#), [yaioxy](#), [yeahChibyke](#), [Zabid27](#), and [zcaj](#)

- `src/WERC7575Vault.sol` [#L434-L437](#)
- `src/WERC7575Vault.sol` [#L464-L467](#)

The `withdraw` and `redeem` functions in `WERC7575Vault` are designed to allow a caller (`msg.sender`) to withdraw assets on behalf of a share owner, adhering to the ERC4626 standard signature. However, the implementation lacks the critical authorization check to verify that `msg.sender` is either the owner themselves or an authorized spender of the owner's shares.

In a standard ERC4626 implementation where the Vault tokenizes the shares (inheriting ERC20), the `withdraw` function calls `_spendAllowance(owner, msg.sender, shares)`,

which verifies and decrements the allowance that the owner has granted to the `msg.sender`. In this WERC7575Vault architecture, the Share Token is a separate contract (`_shareToken`). The `_withdraw` internal function calls `_shareToken.spendSelfAllowance(owner, shares)`. Given that this is an external call from the Vault, this function presumably checks and consumes the allowance that the owner has granted to the Vault (`address(this)`), ensuring the Vault has permission to burn the shares.

Crucially, **no check exists to verify the relationship between `msg.sender` and `owner`**.

1. There is no `msg.sender == owner` check.
2. There is no check of `allowance[owner][msg.sender]` on the Share Token.
3. Even if the Vault attempted to check `allowance[owner][msg.sender]`, it could not securely decrement it via standard ERC20 functions (as only the spender can decrement their own allowance via `transfer/spend`).

As a result, if an owner has approved the Vault contract (which is likely a prerequisite for the Vault to burn shares during a legitimate withdrawal), **any malicious actor** can call `withdraw(..., ..., owner)` or `redeem(..., ..., owner)`, burn the owner's shares, and direct the underlying assets to themselves.

Impact

This is a critical vulnerability allowing the theft of user funds. An attacker can drain the assets associated with any user who has authorized the Vault to interact with their shares. If the system is configured such that the Vault has privileged "burn" rights (requiring no user approval), then an attacker can drain the funds of **all** share owners.

Recommendation

1. **Immediate Mitigation:** Enforce that the caller is the owner of the shares.

```
function withdraw(uint256 assets, address receiver, address owner)
public nonReentrant whenNotPaused returns (uint256 shares) {
    if (msg.sender != owner) revert
    IERC20Errors.ERC20InvalidSender(msg.sender);
    shares = previewWithdraw(assets);
    _withdraw(assets, shares, receiver, owner);
}
```

(Apply similarly to `redeem`).

2. **Protocol Upgrade:** If the business requirement is to support third-party withdrawals (standard ERC4626 allowance functionality), the WERC7575ShareToken interface must be upgraded to support a secure `spendAllowance(address owner, address`

spender, uint256 amount) function that allows a trusted Vault to verify and consume the allowance granted by an owner to a spender (msg.sender).

[View detailed Proof of Concept](#)

Medium Risk Findings (3)

[M-01] The unregistering of vaults can be DoSed by a malicious user.

Submitted by [dimulski](#), also found by [Orpse](#), [Ox_oj](#), [OxApple](#), [OxD4n13l](#), [Oxdoichantran](#), [Oxhunter20](#), [Oxlucky](#), [OxMafiaBug](#), [Oxnightswatch](#), [OxSecurious](#), [Aamir](#), [aestheticbhai](#), [ahahaHard1k](#), [alOx23](#), [ameng](#), [ami](#), [anchabadze](#), [Audittens](#), [Avalance](#), [Bala1796](#), [blutorque](#), [Bobai23](#), [cOpp3rscr3w3r](#), [cheng9061](#), [cosin3](#), [cpsec](#), [cryptolyndon](#), [demaxl](#), [Ekene](#), [farismaulana](#), [Flare](#), [fullstop](#), [gabkov](#), [Gakarot](#), [glorbo](#), [h2134](#), [hezze](#), [hgrano](#), [holtzzx](#), [igbinosuneric](#), [j3x](#), [jasonxiale](#), [JoshuaM33](#), [Keepitfortoby](#), [khaye26](#), [KuwaTakushi](#), [lazlosoot](#), [lodelux](#), [MadSisyphus](#), [mahdifa](#), [makarov](#), [Mathriel](#), [meshaqRapha](#), [mrMorningstar](#), [MysteryAuditor](#), [Nyxaris](#), [oldguard](#), [orangesantra](#), [osok](#), [phoenixV110](#), [PotEater](#), [prk0](#), [romans](#), [s4bot3ur](#), [Sabit](#), [saraswati](#), [SarveshLimaye](#), [ScarletFir](#), [ShaedyWays](#), [skipper](#), [Sneks](#), [SOPROBRO](#), [sri](#), [Tarnished](#), [TheWitchDoctor](#), [trachev](#), [Uddercover](#), [Utsav](#), [Valves](#), [whitehair0330](#), [Willy_Petrus](#), [Wojack](#), [x86sgn](#), [xAlismx](#), and [ZafiN](#)

- src/ShareTokenUpgradeable.sol [#L282](#)
- src/WERC7575ShareToken.sol [#L256](#)

Both the ShareTokenUpgradeable.sol and the WERC7575ShareToken.sol contracts have mechanisms to remove vaults in order to unlink them from the share token. Both contracts allow a maximum of 10 vaults to be registered. The implementation of the unregisterVault() function is different in both of the contracts, however in both instances the unregistering of a vault can be DoSed.

In the ShareTokenUpgradeable.sol the unregisterVault() function performs several checks:

```
function unregisterVault(address asset) external onlyOwner {
    if (asset == address(0)) revert ZeroAddress();
    ShareTokenStorage storage $ = _getShareTokenStorage();

    (bool exists, address vaultAddress) =
$.assetToVault.tryGet(asset);
    if (!exists) revert AssetNotRegistered();

    // COMPREHENSIVE SAFETY CHECK: Ensure vault has no user funds at
risk
```

```

        // This covers pending deposits, claimable redemptions, ERC7887
        cancelations, and any remaining assets

        // 1. Check vault metrics for pending requests, active users, and
        ERC7887 cancelation assets
        try IVaultMetrics(vaultAddress).getVaultMetrics() returns
        (IVaultMetrics.VaultMetrics memory metrics) {
            if (metrics.isActive) revert CannotUnregisterActiveVault();
            if (metrics.totalPendingDepositAssets != 0) {
                revert CannotUnregisterVaultPendingDeposits();
            }
            if (metrics.totalClaimableRedeemAssets != 0) {
                revert CannotUnregisterVaultClaimableRedemptions();
            }
            if (metrics.totalCancelDepositAssets != 0) {
                revert CannotUnregisterVaultAssetBalance();
            }
            if (metrics.activeDepositRequestersCount != 0) {
                revert CannotUnregisterVaultActiveDepositRequesters();
            }
            if (metrics.activeRedeemRequestersCount != 0) {
                revert CannotUnregisterVaultActiveRedeemRequesters();
            }
        } catch {
            // If we can't get vault metrics, we can't safely verify no
            pending requests
            revert CannotUnregisterActiveVault();
        }
        // 2. Final safety: Check raw asset balance in vault contract
        // This catches any remaining assets including investments and
        edge cases
        // If this happens, there is either a bug in the vault
        // or assets were sent to the vault without directly
        if (IERC20(asset).balanceOf(vaultAddress) != 0) {
            revert CannotUnregisterVaultAssetBalance();
        }

        // Remove vault registration (automatically removes from
        enumerable collection)
        $.assetToVault.remove(asset);
        delete $.vaultToAsset[vaultAddress];

        emit VaultUpdate(asset, address(0));
    }

```

As seen from the code snippet above, there are multiple ways a malicious actor can brick the removal of a vault. He can always frontrun the call to the `unregisterVault()` function and transfer 1 WEI to the `ERC7575VaultUpgradeable.sol` contract implementation that the owner is trying to unregister.

Additionally, he can always leave 1 WEI of assets that he doesn't redeem in the `totalClaimableRedeemAssets` additionally that will keep the `activeRedeemRequestersCount` parameter bigger than 0. Similarly malicious depositors can keep 1 WEI in their respective `claimableDepositAssets` mapping and this way they won't be removed from the `activeDepositRequesters` enumerable set, and the `activeDepositRequestersCount` will be bigger than 0.

In the `WERC7575ShareToken.sol` contract, the `unregisterVault()` function checks the asset balance of the specified vault. The `WERC7575Vault.sol` contract has a pausing mechanism that prevents the user from depositing and withdrawing if the contract is paused. However there is no functionality that prevents a user from transferring 1 WEI of the vault asset to the contract. There are no mechanisms that allow a trusted role to remove any dust amounts from the contract. A user can always transfer 1 WEI to the `WERC7575Vault.sol` contract in order to DoS the call to `unregisterVault()` function.

Additionally, the contracts are to be deployed on Berachain where malicious actors can always frontrun the call to the `unregisterVault()` function.

If there are already 10 vaults associated with the share token, and users are not utilizing 1 vault, the admin won't be able to remove that vault and register another vault in its place. This breaks a core contract functionality.

Recommended mitigation steps

To fix the problem with `unregisterVault()` in the `ShareTokenUpgradeable.sol` contract, in the `ERC7575VaultUpgradeable.sol` contract, consider adding admin modifier to the `deposit()`, `mint()`, `redeem()`, `withdraw()`, `claimCancelDepositRequest()` and `claimCancelRedeemRequest()` functions in order to run those functions for the respective users. Additionally consider adding an admin function that can set the `totalClaimableRedeemAssets` and the `totalPendingDepositAssets` parameters to 0, if there are some dust amounts in them. Or change the `ShareTokenUpgradeable::unregisterVault()` to allow a vault to be unregistered even if there are some dust amounts left.

To fix the problem with `unregisterVault()` in the `WERC7575ShareToken.sol` contract, consider adding an admin function that can withdraw dust amounts from the `WERC7575Vault.sol` contract.

[M-02] Stale Redemption Liabilities Lead to Leveraged Losses for Remaining Shareholders and Potential Denial of Service

Submitted by [farismaulana](#), also found by [Audittens](#), [Ekene](#), [h2134](#), [jo13](#), [Noro](#), and [Valves](#)

- [src/ERC7575VaultUpgradeable.sol #L831](#)
- [src/ERC7575VaultUpgradeable.sol #L1208](#)
- [src/ShareTokenUpgradeable.sol #L369-L390](#)
- [src/ShareTokenUpgradeable.sol #L603-L620](#)

When a user request redeem and then fulfilled by the investment manager, there is no expiration or repricing happening. The function calculates the PPS (price-per-share) at the time `fulfillRedeem` is called but there is no obligation for the user to immediately call `redeem` or `withdraw` to execute the redemption because of the way `async` functionality is implemented.

[ERC7575VaultUpgradeable.sol#L822C1-L837C84](#)

```
function fulfillRedeem(address controller, uint256 shares) public
nonReentrant returns (uint256 assets) {
    ...
    @>    assets = _convertToAssets(shares, Math.Rounding.Floor);

    $.pendingRedeemShares[controller] -= shares;
    $.claimableRedeemAssets[controller] += assets;
    $.claimableRedeemShares[controller] += shares;
    $.totalClaimableRedeemAssets += assets;
    $.totalClaimableRedeemShares += shares; // Track shares that will
    be burned
```

Notice that in the snippet above, the `_convertToAssets` would be called to calculate how many assets the user would have using PPS at the time that the function is called.

If we dig deeper, the function would go into

`ShareTokenUpgradeable::getCirculatingSupplyAndAssets` to determine how much shares supply and total assets are used in the `_convertToAssets`.

Assets that are still in `rBalance` also would be used as an asset.

```
function _calculateInvestmentAssets() internal view returns (uint256
totalInvestmentAssets) {
    ...
    // Get our balance of investment ShareToken (already normalized
    to 18 decimals)
```



```

        totalInvestmentAssets =
IERC20(investmentShareToken).balanceOf(address(this));

        // Add rBalanceOf (reserved balance) if the investment share
token supports it
        try
IERERC7575ShareToken(investmentShareToken).rBalanceOf(address(this))
returns (uint256 rShares) {
@>         totalInvestmentAssets += rShares;
        } catch {
            // If rBalanceOf is not supported, continue with regular
balance only
        }
    }
}

```

If the investment is in profit and some user redeem is fulfilled, the PPS used would be high.

The issue is that profit is not certain, there are fluctuations. If at first month the deal profited 10% and some user redeem request fulfilled, they would get good PPS even though they didn't execute the redemption and basically their asset is still in the vault. The next month, if there is loss and making the initial investment amount is at 2% profit cumulatively, the last user will not be able to realize the fair value of their shares.

To clearly show the scenario, we can look at the issue like this:

1. A user requests redemption during a profitable period (High PPS).
2. The Manager fulfills the request, locking in the high payout value.
3. The user can delay the final withdraw execution or execute it right away. Fulfilled user is protected.
4. If the vault subsequently suffers a loss (drawdown), the fulfilled user is protected, as their claim is now a fixed value from step 2.
5. This forces the remaining shareholders to absorb 100% of the loss (leveraged loss), potentially leading to vault insolvency where $TotalAssets < TotalClaimableLiabilities$.

The impact for this issue is:

1. A race condition for executing withdrawing a fulfillment, because the last user would be unable to fully redeem their full shares.
2. Thus leading to a potential insolvency.

Recommended mitigation steps

There are a few recommended options:

1. The final redeem or withdraw should be repriced at the PPS value and not locked at fulfillment call.

2. Make the fulfill function instantly send the asset to requester, or make the fulfill have expired time so it needs to be executed immediately by user. This still respects the async functionality.
3. Localize each rBalance change per epoch, so fulfillment is using the same rate for everyone. Making it per epoch would ensure each request and fulfillment has the same containerized asset and rBalance profit/loss does not overlap each other.

[View detailed Proof of Concept](#)

[SukukFi disputed and commented:](#)

We understand your point regarding the "profit cancellation" and how it impacts the remaining users. However, we consider this to be the strict, intended behavior dictated by the EIP-7540 (Asynchronous Tokenized Vaults) standard, which our protocol implements.

The architecture of EIP-7540 relies entirely on the clear separation between the Pending state and the Claimable state:

1. The Static Nature of Claimable Assets Under EIP-7540, once a redemption request transitions to the Claimable state, the share-to-asset conversion is finalized. At that exact moment, the user is owed a *fixed, static* amount of assets. Because they are fully removed from the vault's risk pool, they no longer earn yield, but symmetrically, they are strictly shielded from any subsequent NAV drops or "profit cancellations."
2. Repricing Violates the Standard The auditor's core suggestion—that the final redeem() or withdraw() should be repriced at execution rather than locked at fulfillment—would explicitly violate EIP-7540. If a user's Claimable balance continued to float with the vault's NAV, there would be no functional economic distinction between the Pending and Claimable states. The entire purpose of the async standard would be defeated.
3. Standard Async Mechanics What the finding identifies as a vulnerability is simply the reality of an asynchronous state transition. Moving from a risk-on, yield-bearing state to a risk-off, static state mandates that subsequent market fluctuations (including profit cancellations) only affect those remaining in the risk pool. This is the fundamental trade-off of the EIP-7540 lifecycle.

Because our protocol is built to strictly adhere to the EIP-7540 specification, mitigating this by repricing claimable assets or forcing instant execution would break our compliance with the standard.

We will gladly follow your suggestion and mark this finding as Acknowledged, treating it as an accepted design mechanic mandated by the ERC standard.

[\[M-03\]_fulfillRedeem may corrupt share price and allow over-redemption of assets](#)

Submitted by [Audittens](#), also found by [OxGOpI](#), [OxSecurious](#), [OxSeer](#), [blockace](#), [dee24](#), [Destiny_rs](#), [EPSec](#), [s4bot3ur](#), and [slvDev](#)

- `src/ShareTokenUpgradeable.sol` [#L710](#)
- `src/ShareTokenUpgradeable.sol` [#L378-L389](#)
- `src/ERC7575VaultUpgradeable.sol` [#L1533-L1535](#)
- `src/ERC7575VaultUpgradeable.sol` [#L1176-L1179](#)
- `src/ERC7575VaultUpgradeable.sol` [#L822-L841](#)

The share price of `ShareTokenUpgradeable` is computed as:

- $\text{sharePrice} \approx \text{normalizedAssets} / \text{circulatingSupply}$
- $\text{normalizedAssets} = \sum \max(0, \text{asset.balanceOf(vault)} - \text{pendingDepositAssets} - \text{claimableRedeemAssets} - \text{cancelDepositAssets}) + \text{investmentAssets}$
- $\text{circulatingSupply} = \text{supply} - \sum \text{claimableRedeemShares}$

Typically this calculation is correct. However, there is an edge case: when $\text{asset.balanceOf(vault)} < \text{pendingDepositAssets} + \text{claimableRedeemAssets} + \text{cancelDepositAssets}$, expression $\text{asset.balanceOf(vault)} - \text{pendingDepositAssets} - \text{claimableRedeemAssets} - \text{cancelDepositAssets}$ is negative, so [max\(0, ...\)](#) evaluates to 0. In this case, `normalizedAssets` is larger than it should be but `circulatingSupply` is correct. Therefore, **share price becomes inflated**.

This edge case appears when investment manager fulfills redeem, but assets existing directly in this vault balance are not enough to cover it (because many assets are in other vaults or in investment).

Misbehavior scenario

1. Alice and Bob deposit 1000 assets each.
2. Investment manager invests these 2000 assets.
3. Alice and Bob request redeeming all their shares.
4. Investment manager fulfills Alice's redeem.
5. Now:

```
asset.balanceOf(vault) = 0
claimableRedeemAssets = 1000
investmentAssets = 2000
normalizedAssets = max(0, 0 - 1000) + 2000 = max(0, -1000) + 2000 =
0 + 2000 = 2000
-----
supply = 2000
claimableRedeemShares = 1000
circulatingSupply = 2000 - 1000 = 1000
-----
```

```
sharePrice = normalizedAssets / circulatingSupply = 2000 / 1000 = 2
instead of 1
```

6. Investment manager fulfills Bob's redeem, and Bob gets more assets because of inflated share price.
7. Investment manager withdraws all investment.
8. Now Bob completes redeem, and gets more assets than he should.

Impact

Inflated share price causes direct loss of funds:

- Depositors will receive less shares than expected;
- Some redeemers will receive more assets than expected;
- When all assets are exhausted, other redeemers will receive nothing.

Recommended mitigation steps

Either:

- Make `ERC7575VaultUpgradeable.totalAssets()` return `int256` instead of `uint256` and remove the `balance > reservedAssets` [condition](#); or
- In `ERC7575VaultUpgradeable.fulfillRedeem()`, check that `assets <= totalAssets()`.

[View detailed Proof of Concept](#)

Low Risk and Informational Issues

For this audit, 54 QA reports were submitted by wardens compiling low risk and informational issues. The [QA report highlighted below](#) by `prk0` received the top score from the judge. 22 Low-severity findings were also submitted individually, and can be viewed [here](#).

The following wardens also submitted QA reports: [Oxbereket](#), [Oxchrispiao](#), [Oxheartcode](#), [Oxki](#), [Oxnightswatch](#), [Oxnija](#), [Oxscater](#), [Oxsh](#), [aestheticbhai](#), [Ahmerdrarerh](#), [Almanax](#), [ami](#), [asotu3](#), [Avalance](#), [blutorque](#), [boodieboodieboo](#), [Brene](#), [cosin3](#), [Diavolo](#), [Ekene](#), [elyas6126](#), [Farnad](#), [freebird0323](#), [grey](#), [h2134](#), [holtzxx](#), [home1344](#), [jerry0422](#), [K42](#), [Manvita](#), [mbuba666](#), [mrMorningstar](#), [mzfr](#), [nutledger](#), [orangesantra](#), [Oxhsn](#), [phoenixV110](#), [Polaris_Snowfall](#), [redfox](#), [rubencrxz](#), [s4bot3ur](#), [SarveshLimaye](#), [Sathish9098](#), [slvDev](#), [SnowX](#), [SOPROBRO](#), [Sparrow](#), [v12](#), [vangrim](#), [Willy_Petrus](#), [yaoxy](#), [YF1](#), and [Yuubee](#).

[01] Vault approval is not revoked in

`ShareTokenUpgradeable::unregisterVault()` **flow**

```

function registerVault(address asset, address vaultAddress) external
onlyOwner {
    // SNIP

    // If investment ShareToken is already configured, set up investment
for the new vault
    // Only configure if the vault address is a deployed contract
> address investmentShareToken = $.investmentShareToken;
> if (investmentShareToken != address(0)) {
>     _configureVaultInvestmentSettings(asset, vaultAddress,
investmentShareToken);
> }

    // If investment manager is already configured, set it for the new
vault
    // Only configure if the vault address is a deployed contract
    address investmentManager = $.investmentManager;
    if (investmentManager != address(0)) {

ERC7575VaultUpgradeable(vaultAddress).setInvestmentManager(investmentMana
ger);
    }

    emit VaultUpdate(asset, vaultAddress);
}

function _configureVaultInvestmentSettings(address asset, address
vaultAddress, address investmentShareToken) internal {
    // Find the corresponding investment vault for this asset
    address investmentVaultAddress =
IERC7575ShareExtended(investmentShareToken).vault(asset);

    // Configure investment vault if there's a matching one for this
asset
    if (investmentVaultAddress != address(0)) {

ERC7575VaultUpgradeable(vaultAddress).setInvestmentVault(IERC7575(investm
entVaultAddress));

        // Grant unlimited allowance to the vault on the investment
ShareToken
> IERC20(investmentShareToken).approve(vaultAddress,
type(uint256).max);
    }
}

```

`_configureVaultInvestmentSettings()` will be called in the `registerVault()` flow, if `investmentShareToken` has been configured.

In `_configureVaultInvestmentSettings()`, the `vaultAddress` is configured with max allowance for `investmentShareToken`.

The allowance is not reset in the `unregisterVault()` flow, which will leave a lingering max approval to the old vault.

Impact

Lingering `investmentShareToken` allowance on previous vault even after vault update.

Recommendation

```
// ShareTokenUpgradeable.sol
function unregisterVault(address asset) external onlyOwner {
    // SNIP

    // Remove vault registration (automatically removes from enumerable
    collection)
    $.assetToVault.remove(asset);
    delete $.vaultToAsset[vaultAddress];

+   IERC20(investmentShareToken).approve(vaultAddress, 0);

    emit VaultUpdate(asset, address(0));
}
```

[02] ShareTokenUpgradeable and WERC7575ShareToken are non-compliant with expected ERC-7575 Standard due to incorrect `supportsInterface()` implementation

ERC-165 support

Vaults implementing ERC-7575 MUST implement the ERC-165 `supportsInterface` function.

The Vault contract MUST return the constant value `true` if `0x2f0a18c5` is passed through the `interfaceID` argument.

The share contract SHOULD implement the ERC-165 `supportsInterface` function.

The share token MUST return the constant value `true` if `0xf815c03d` is passed through the `interfaceID` argument.

ref: <https://eips.ethereum.org/EIPS/eip-7575>

```
// ShareTokenUpgradeable.sol
function supportsInterface(bytes4 interfaceId) public view virtual
override returns (bool) {
    return interfaceId == type(IERC7575ShareExtended).interfaceId ||
interfaceId == type(IERC7540Operator).interfaceId || interfaceId ==
type(IERC165).interfaceId;
}

// WERC7575ShareToken.sol
function supportsInterface(bytes4 interfaceId) public view virtual
override(ERC165) returns (bool) {
    return interfaceId == type(IERC7575Share).interfaceId ||
super.supportsInterface(interfaceId);
}
```

ShareTokenUpgradeable (IERC7575ShareExtended) and WERC7575ShareToken (IERC7575Share) both implement supportsInterface() as recommended by ERC-7575.

It is also stated that supportsInterface() MUST return true for input: 0xf815c03d - This case is not handled in supportsInterface() for both share token implementations, which makes these implementations non-compliant with ERC-7575 Specification.

Impact

ShareTokenUpgradeable and WERC7575ShareToken are non-compliant with expected ERC-7575 Standard.

Recommendation

```
// ShareTokenUpgradeable.sol
function supportsInterface(bytes4 interfaceId) public view virtual
override returns (bool) {
-   return interfaceId == type(IERC7575ShareExtended).interfaceId ||
interfaceId == type(IERC7540Operator).interfaceId || interfaceId ==
type(IERC165).interfaceId;
+   return interfaceId == type(IERC7575ShareExtended).interfaceId ||
interfaceId == type(IERC7540Operator).interfaceId || interfaceId ==
type(IERC165).interfaceId || interfaceId == 0xf815c03d;
}

// WERC7575ShareToken.sol
```

```

function supportsInterface(bytes4 interfaceId) public view virtual
override(ERC165) returns (bool) {
-   return interfaceId == type(IERC7575Share).interfaceId ||
super.supportsInterface(interfaceId);
+   return interfaceId == type(IERC7575Share).interfaceId ||
super.supportsInterface(interfaceId) || interfaceId == 0xf815c03d;
}

```

[03] Pre-increment accountsLength in

WERC7575ShareToken::consolidateTransfers()

```

function consolidateTransfers(
    address[] calldata debtors,
    address[] calldata creditors,
    uint256[] calldata amounts
)
    internal
    pure
    returns (DebitAndCredit[] memory accounts, uint256 accountsLength)
{
    // SNIP

    // Outer loop: process each transfer
    for (uint256 i = 0; i < debtorsLength;) {
        address debtor = debtors[i];
        address creditor = creditors[i];
        uint256 amount = amounts[i];

        // Skip self-transfers (debtor == creditor)
        if (debtor != creditor) {
            // SNIP

            // Create new account entries only if not found in existing
accounts
            if ((addFlags & 1) != 0) {
                // Check bit 0 (addDebtor)
                accounts[accountsLength] = DebitAndCredit(debtor, amount,
0);
            >
                accountsLength++; // @audit pre-increment
            }
            if ((addFlags & 2) != 0) {
                // Check bit 1 (addCreditor)
                accounts[accountsLength] = DebitAndCredit(creditor, 0,
amount);
            >
                accountsLength++;

```



```

    }
  }

  unchecked {
    ++i;
  }
}
}

```

accountsLength can be pre-incremented for a small amount of gas savings and to keep the system consistent.

Recommendation

```

function consolidateTransfers(
  address[] calldata debtors,
  address[] calldata creditors,
  uint256[] calldata amounts
)
  internal
  pure
  returns (DebitAndCredit[] memory accounts, uint256 accountsLength)
{
  // SNIP

  // Outer loop: process each transfer
  for (uint256 i = 0; i < debtorsLength;) {
    address debtor = debtors[i];
    address creditor = creditors[i];
    uint256 amount = amounts[i];

    // Skip self-transfers (debtor == creditor)
    if (debtor != creditor) {
      // SNIP

      // Create new account entries only if not found in existing
accounts
      if ((addFlags & 1) != 0) {
        // Check bit 0 (addDebtor)
        accounts[accountsLength] = DebitAndCredit(debtor, amount,
0);
-         accountsLength++; // @audit pre-increment
+         ++accountsLength;
      }
      if ((addFlags & 2) != 0) {

```

```

        // Check bit 1 (addCreditor)
        accounts[accountsLength] = DebitAndCredit(creditor, 0,
amount);
-         accountsLength++;
+         ++accountsLength;
    }
}

unchecked {
    ++i;
}
}
}

```

[04] Deposit event deviates from ERC-7540 Specification

deposit and mint overloaded methods

Implementations MUST support an additional overloaded deposit and mint method on the specification from [ERC-4626](#), with an additional controller input of type address:

- deposit(uint256 assets, address receiver, address controller)
- mint(uint256 shares, address receiver, address controller)

Calls MUST revert unless msg.sender is either equal to controller or an operator approved by controller.

The controller field is used to discriminate the Request for which the assets should be claimed in the case where msg.sender is NOT controller.

When the Deposit event is emitted, the first parameter MUST be the controller, and the second parameter MUST be the receiver.

ref: <https://eips.ethereum.org/EIPS/eip-7540>

```

// ERC7575VaultUpgradeable.sol
function deposit(uint256 assets, address receiver, address controller)
public nonReentrant returns (uint256 shares) {
    // SNIP

    > emit Deposit(receiver, controller, assets, shares);

    // Transfer shares from vault to receiver using ShareToken
    if (!IERC20Metadata($.shareToken).transfer(receiver, shares)) {
        revert ShareTransferFailed();
    }
}

```

```
}
```

It is stated that the `Deposit` event should emit `controller` as the first parameter and `receiver` as the second. The implementation currently does the opposite.

Recommendation

```
function deposit(uint256 assets, address receiver, address controller)
public nonReentrant returns (uint256 shares) {
    // SNIP

-   emit Deposit(receiver, controller, assets, shares);
+   emit Deposit(controller, receiver, assets, shares);

    // Transfer shares from vault to receiver using ShareToken
    if (!IERC20Metadata($.shareToken).transfer(receiver, shares)) {
        revert ShareTransferFailed();
    }
}

function mint(uint256 shares, address receiver, address controller)
public nonReentrant returns (uint256 assets) {
    // SNIP

-   emit Deposit(receiver, controller, assets, shares);
+   emit Deposit(controller, receiver, assets, shares);

    // Transfer shares from vault to receiver using ShareToken
    if (!IERC20Metadata($.shareToken).transfer(receiver, shares)) {
        revert ShareTransferFailed();
    }
}
```

[O5] Several events deviate from ERC-7887 Specification

```
// ERC7575VaultUpgradeable.sol
function cancelDepositRequest(uint256 requestId, address controller)
external nonReentrant {
    // SNIP

    emit CancelDepositRequest(controller, controller, REQUEST_ID,
msg.sender, pendingAssets);
    // @audit does not match ERC7887 specification
}
```

```

        // CancelDepositRequest(controller, requestId, sender)
    }

    function claimCancelDepositRequest(uint256 requestId, address receiver,
    address controller) external nonReentrant {
        // SNIP

        // Event emission
        emit CancelDepositRequestClaimed(controller, receiver, REQUEST_ID,
assets);
        // @audit does not match ERC7887 specification
        // CancelDepositClaim(controller, receiver, requestId, sender,
assets)
    }

    function cancelRedeemRequest(uint256 requestId, address controller)
    external nonReentrant {
        // SNIP

        emit CancelRedeemRequest(controller, controller, REQUEST_ID,
msg.sender, pendingShares);
        // @audit does not match ERC7887 specification
        // CancelRedeemRequest(controller, requestId, sender)
    }

    function claimCancelRedeemRequest(uint256 requestId, address owner,
    address controller) external nonReentrant {
        // SNIP

        // Event emission
        emit CancelRedeemRequestClaimed(controller, owner, REQUEST_ID,
shares);
        // @audit deviates from ERC7887 specification
        // CancelRedeemClaim(controller, receiver, requestId, sender, shares)
    }

```

ref: <https://eips.ethereum.org/EIPS/eip-7887>

There are several functions related to cancellation logic, which do not emit events, as defined in the ERC-7887 Specification.

[O6] Input parameter, `receiver`, is not checked in several methods

```

function withdraw(uint256 assets, address receiver, address controller)
public nonReentrant returns (uint256 shares) {

```

```

VaultStorage storage $ = _getVaultStorage();
if (!(controller == msg.sender ||
IERC7540($.shareToken).isOperator(controller, msg.sender))) {
    revert InvalidCaller();
}
if (assets == 0) revert ZeroAssets();

uint256 availableAssets = $.claimableRedeemAssets[controller];
if (assets > availableAssets) revert InsufficientClaimableAssets();

// Calculate proportional shares for the requested assets
uint256 availableShares = $.claimableRedeemShares[controller];
shares = assets.mulDiv(availableShares, availableAssets,
Math.Rounding.Floor);

if (shares == availableShares) {
    // Remove from active redeem requesters if no more claimable
    assets and the potential dust
    $.activeRedeemRequesters.remove(controller);
    delete $.claimableRedeemAssets[controller];
    delete $.claimableRedeemShares[controller];
} else {
    $.claimableRedeemAssets[controller] -= assets;
    $.claimableRedeemShares[controller] -= shares;
}

$.totalClaimableRedeemAssets -= assets;
$.totalClaimableRedeemShares -= shares; // Decrement shares that are
being burned

// Burn the shares as per ERC7540 spec - shares are burned when
request is claimed
if (shares > 0) {
    ShareTokenUpgradeable($.shareToken).burn(address(this), shares);
}

> emit Withdraw(msg.sender, receiver, controller, assets, shares);

> SafeTokenTransfers.safeTransfer($.asset, receiver, assets);
}

```

The receiver parameter is never checked against address(0) - It is recommended to remove these “user foot-guns.”

Recommendation

Apply the changes below to ERC7575VaultUpgradeable - deposit(), mint(), redeem(), and withdraw().

```
function withdraw(uint256 assets, address receiver, address controller)
public nonReentrant returns (uint256 shares) {
    VaultStorage storage $ = _getVaultStorage();
    if (!(controller == msg.sender ||
IERC7540($.shareToken).isOperator(controller, msg.sender))) {
        revert InvalidCaller();
    }
    if (assets == 0) revert ZeroAssets();
+   if (receiver == address(0)) revert InvalidReceiver();

    // SNIP
}
```

[07] balanceOf() check is redundant

```
function requestDeposit(uint256 assets, address controller, address
owner) external nonReentrant returns (uint256 requestId) {
    VaultStorage storage $ = _getVaultStorage();
    if (!$.isActive) revert VaultNotActive();
    if (!(owner == msg.sender || IERC7540($.shareToken).isOperator(owner,
msg.sender))) revert InvalidOwner();
    if (assets == 0) revert ZeroAssets();
    if (assets < $.minimumDepositAmount * (10 ** $.assetDecimals)) {
        revert InsufficientDepositAmount();
    }
    > uint256 ownerBalance = IERC20Metadata($.asset).balanceOf(owner);
    > if (ownerBalance < assets) {
    >     revert ERC20InsufficientBalance(owner, ownerBalance, assets);
    > }
    // ERC7887: Block new deposit requests while cancelation is pending
for this controller
    if ($.controllersWithPendingDepositCancelations.contains(controller))
    {
        revert DepositCancelationPending();
    }

    // Pull-Then-Credit pattern: Transfer assets first before updating
state
    // This ensures we only credit assets that have been successfully
received
```

```

    // Protects against transfer fee tokens and validates the actual
    amount transferred
    > SafeTokenTransfers.safeTransferFrom($.asset, owner, address(this),
    assets);

    // State changes after successful transfer
    $.pendingDepositAssets[controller] += assets;
    $.totalPendingDepositAssets += assets;
    $.activeDepositRequesters.add(controller);

    // Event emission
    emit DepositRequest(controller, owner, REQUEST_ID, msg.sender,
    assets);
    return REQUEST_ID;
}

```

In `ERC7575VaultUpgradeable::requestDeposit()`, there is a balance check that occurs prior to the underlying token being pulled from `msg.sender`.

This check is redundant and can be safely removed because a balance check already occurs in the `transferFrom()` call.

Recommendation

Apply the changes below to `requestDeposit()` and `requestRedeem()`

```

function requestDeposit(uint256 assets, address controller, address
owner) external nonReentrant returns (uint256 requestId) {
    VaultStorage storage $ = _getVaultStorage();
    if (!$.isActive) revert VaultNotActive();
    if (!(owner == msg.sender || IERC7540($.shareToken).isOperator(owner,
msg.sender))) revert InvalidOwner();
    if (assets == 0) revert ZeroAssets();
    if (assets < $.minimumDepositAmount * (10 ** $.assetDecimals)) {
        revert InsufficientDepositAmount();
    }
    - uint256 ownerBalance = IERC20Metadata($.asset).balanceOf(owner);
    - if (ownerBalance < assets) {
    -     revert ERC20InsufficientBalance(owner, ownerBalance, assets);
    - }
    // ERC7887: Block new deposit requests while cancelation is pending
    for this controller
    if ($.controllersWithPendingDepositCancelations.contains(controller))
    {
        revert DepositCancelationPending();
    }
}

```

```

    }

    // SNIP
}

```

[08] scalingFactor check can be removed

```

function initialize(IERC20Metadata asset_, address shareToken_, address
owner) public initializer {
    if (shareToken_ == address(0)) {
        revert IERC20Errors.ERC20InvalidReceiver(address(0));
    }
    if (address(asset_) == address(0)) {
        revert IERC20Errors.ERC20InvalidSender(address(0));
    }

    // Validate asset compatibility and get decimals
    uint8 assetDecimals;
    try IERC20Metadata(address(asset_)).decimals() returns (uint8
decimals) {
        if (decimals < DecimalConstants.MIN_ASSET_DECIMALS || decimals >
DecimalConstants.SHARE_TOKEN_DECIMALS) {
            revert UnsupportedAssetDecimals();
        }
        assetDecimals = decimals;
    } catch {
        revert AssetDecimalsFailed();
    }

    // Validate share token compatibility and enforce 18 decimals
    try IERC20Metadata(shareToken_).decimals() returns (uint8 decimals) {
        if (decimals != DecimalConstants.SHARE_TOKEN_DECIMALS) {
            revert WrongDecimals();
        }
    } catch {
        revert AssetDecimalsFailed();
    }

    __Ownable_init(owner);

    VaultStorage storage $ = _getVaultStorage();
    $.asset = address(asset_);
    $.assetDecimals = assetDecimals;
    $.shareToken = shareToken_;
    $.investmentManager = owner; // Initially owner is investment manager
}

```



```

$.isActive = true; // Vault is active by default

// Calculate scaling factor for decimal normalization: 10^(18 -
assetDecimals)
uint256 scalingFactor = 10 ** (DecimalConstants.SHARE_TOKEN_DECIMALS
- assetDecimals);
> if (scalingFactor > type(uint64).max) revert ScalingFactorTooLarge();
$.scalingFactor = uint64(scalingFactor);
$.minimumDepositAmount = 1000;
}

```

Due to the checks for shareToken_ decimals and asset_ decimals, scalingFactor is always guaranteed to be in the range of: 1 → 1e12

Since we can be sure that scalingFactor is contained within this range, the upper bounds check for scalingFactor can be safely removed.

Recommendation

```

function initialize(IERC20Metadata asset_, address shareToken_, address
owner) public initializer {
    // SNIP

    VaultStorage storage $ = _getVaultStorage();
    $.asset = address(asset_);
    $.assetDecimals = assetDecimals;
    $.shareToken = shareToken_;
    $.investmentManager = owner; // Initially owner is investment manager
    $.isActive = true; // Vault is active by default

    // Calculate scaling factor for decimal normalization: 10^(18 -
assetDecimals)
    uint256 scalingFactor = 10 ** (DecimalConstants.SHARE_TOKEN_DECIMALS
- assetDecimals);
    if (scalingFactor > type(uint64).max) revert ScalingFactorTooLarge();
    $.scalingFactor = uint64(scalingFactor);
    $.minimumDepositAmount = 1000;
}

```

[09] maxMint

It is stated in EIP-7575 that: The existing definitions from [ERC-4626] (<https://eips.ethereum.org/EIPS/eip-4626>) apply.

In EIP-4626, view methods prefixed with ‘max,’ such as `maxMint()`, `maxDeposit()`, `maxRedeem()`, `maxWithdraw()`, should account for both global and user-specific limits.

Both `ERC7575VaultUpgradeable` and `WERC7575Vault` have an admin-controlled parameter, `isActive`, which prevents deposits when enabled.

`WERC7575Vault` implements pausing functionality that should be considered in these methods as well.

Recommendation

```
// WERC7575Vault.sol
function maxDeposit(address) public pure returns (uint256) {
    if (isActive || paused()) return 0;
    return type(uint256).max;
}

function maxMint(address) public pure returns (uint256) {
    if (isActive || paused()) return 0;
    return type(uint256).max;
}

function maxWithdraw(address owner) public view returns (uint256) {
    return _convertToAssets(_shareToken.balanceOf(owner),
Math.Rounding.Floor);
}

function maxRedeem(address owner) public view returns (uint256) {
    return _shareToken.balanceOf(owner);
}
```

```
// ERC7575VaultUpgradeable.sol
function maxDeposit(address) public pure returns (uint256) {
    if (isActive) return 0;
    return type(uint256).max;
}

function maxMint(address) public pure returns (uint256) {
    if (isActive) return 0;
    return type(uint256).max;
}
```

Detailed Proofs of Concept for the above-listed Low-severity issues may be viewed [here](#).

Disclosures

C4 audits incentivize the discovery of exploits, vulnerabilities, and bugs in smart contracts. Security researchers are rewarded at an increasing rate for finding higher-risk issues. Audit submissions are judged by a knowledgeable security researcher and disclosed to sponsoring developers. C4 does not conduct formal verification regarding the provided code but instead provides final verification.

C4 does not provide any guarantee or warranty regarding the security of this project. All smart contract software should be used at the sole risk and responsibility of users.